Studio Nx

백엔드 181 — compute_waveform()

2026-10-15

Backend(FastAPI/Python) 181 — `compute_waveform()`

항목	내용
파일	`studionx/backend/app/services/sample_audio_service.py`
작업	S1-07 샘플 라이브러리: 업로드 시 파형 미리보기 데이터 생성 (PRD 02 FR-223, FR-210)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	180 `read_audio_info()` · 182 파형 파일 저장 (예정)

1. 하는 일

파일을 float32로 읽어 채널을 하나로 섞고, 표본을 `buckets` 칸으로 나눠 칸마다 (최솟값, 최댓값)을 냅니다. 화면은 이 값으로 파형을 그립니다.

넣은 것	결과
1초 44100Hz 모노 (무음)	`peaks` 1000칸, 모두 `(0.0, 0.0)`
일정한 0.5, buckets=10	10칸 모두 `(0.5, 0.5)`
440Hz 사인 0.8, buckets=20	첫 칸은 음수와 양수를 모두 가짐, 모든 값이 −1.0 ~ 1.0
왼쪽 +0.5, 오른쪽 −0.5 (스테레오)	`channels=2`, 모두 `(0.0, 0.0)` (평균이 0)
1000표본, buckets=100	칸을 표본 수인 10개로 줄임 (빈 칸 방지)
float 2.0	1.0으로 자름 (clamp)
0.123456	0.1235로 반올림
없는 파일 / 오디오 아님 / 빈 파일	404 / 415 / 400
buckets 0	`ValueError`

2. 언제 쓰나

업로드 흐름에서 파일 저장 다음에 이어집니다: 파일 저장 → `read_audio_info`(180) → `compute_waveform`(181) → JSON 파일로 저장(182) → `samples.waveform_path`.

쓰는 곳	
라이브러리 패널 미리보기	FR-224
타임라인 클립 파형	FR-210
`samples` 테이블	`waveform_path` (DB.md §3.2, JSON/PNG) — **JSON 피크 배열**로 결정했습니다 (2026-10-15 사용자 결정)

3. 코드

import (파일 맨 위, `numpy` 추가)

```
from dataclasses import dataclass
from pathlib import Path
```

```
import numpy as np
import soundfile as sf

from app.core.errors import AppError, ErrorCode

AUDIO_MESSAGE = "Use a WAV, MP3, FLAC, OGG or AIFF audio file."

#: 파형 미리보기의 기본 칸 수(peaks 길이). 화면 폭(보통 1000px 남짓)에 맞춘다.
DEFAULT_BUCKETS = 1000
```

파형 값 묶음

```
@dataclass(frozen=True)
class Waveform:
    """파형 미리보기 데이터. peaks 는 (최솟값, 최댓값) 쌍의 목록이다 (각 -1.0 ~ 1.0).

    peaks 의 길이가 buckets 이고, 채널이 여럿이면 하나로 섞은(평균) 값을 쓴다.
    """

    sample_rate: int
    channels: int
    peaks: tuple[tuple[float, float], ...]
```

함수

```
def compute_waveform(path: str | Path, buckets: int = DEFAULT_BUCKETS) -> Waveform:
    if buckets < 1:
        raise ValueError("buckets must be 1 or more")
    info = read_audio_info(path)
    try:
        data, _ = sf.read(path, dtype="float32", always_2d=True)
    except sf.SoundFileError as exc:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, AUDIO_MESSAGE) from exc
    total = int(data.shape[0])
    if total == 0:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "This audio file has no samples.")
    mono = data.mean(axis=1)
    count = min(buckets, total)
    edges = np.linspace(0, total, count + 1).astype(np.int64)
    peaks: list[tuple[float, float]] = []
    for i in range(count):
        segment = mono[edges[i] : edges[i + 1]]
        low = float(np.clip(segment.min(), -1.0, 1.0))
        high = float(np.clip(segment.max(), -1.0, 1.0))
        peaks.append((round(low, 4), round(high, 4)))
    return Waveform(sample_rate=info.sample_rate, channels=info.channels, peaks=tuple(peaks))
```

문법 풀이

식	뜻
`sf.read(path, dtype="float32", always_2d=True)`	파일 전체를 읽어 2차원 `(표본, 채널)` 배열로 만듭니다. `always_2d`라서 모노도 2차원입니다
`data.mean(axis=1)`	채널 방향으로 평균을 내어 하나로 섞습니다

식	뜻
`np.linspace(0, total, count + 1).astype(np.int64)`	칸 경계(표본 번호)를 고르게 나눠 정수로 만듭니다
`mono[edges[i] : edges[i + 1]]`	i번째 칸에 든 표본들입니다
`np.clip(x, -1.0, 1.0)`	값을 −1.0 ~ 1.0 밖으로 나가지 않게 자릅니다
`round(x, 4)`	소수 넷째 자리에서 반올림합니다
`tuple(...)`	값이 바뀌지 않는 목록입니다 (frozen 값 묶음과 어울립니다)

4. 흐름

1. `buckets`를 확인합니다 (1 미만이면 `ValueError`).
2. `read_audio_info`(180)로 없는 파일(404)·못 읽는 파일(415)을 먼저 가립니다.
3. `sf.read`로 전체를 float32로 읽습니다.
4. 표본이 하나도 없으면 400입니다.
5. 채널을 평균해 하나로 섞습니다.
6. 표본을 `count`칸(= `min(buckets, 표본 수)`)으로 나눠, 칸마다 최솟값·최댓값을 −1 ~ 1로 자르고 반올림합니다.
7. `sample_rate`, `channels`, `peaks`를 담아 돌려줍니다.

5. 왜 이렇게

선택	이유
JSON 피크 배열 (min/max 쌍)	화면에서 확대·축소할 때 그리는 쪽이 값을 골라 쓰기 쉽습니다. PNG는 크기가 고정되어 확대하면 흐려집니다 (2026-10-15 사용자 결정)
칸마다 min/max	피크를 전부 그리면 화면 폭보다 표본이 많아 무의미합니다. 칸의 최솟값·최댓값만 남기면 파형 모양이 유지됩니다
채널을 평균 (모노)	화면에 한 줄로 그리는 미리보기에 충분합니다. 채널별로 나눠 그리는 것은 필요해지면 더합니다
값 범위를 −1.0 ~ 1.0으로 자름	float WAV는 1.0을 넘을 수 있어 그리기 좌표가 넘칩니다. 그리기용이라 −1 ~ 1로 자릅니다
소수 넷째 자리 반올림	그리기에 그 이상 정밀도는 필요 없고 JSON 파일이 작아집니다
표본보다 칸이 많으면 줄임	빈 칸을 만들면 그릴 때 끊깁니다. 칸 수가 표본 수를 넘지 않게 합니다
`buckets`는 `ValueError`	부르는 쪽(서버)이 정하는 값이라 사용자 입력이 아닙니다. 잘못 부르면 설정 실수이므로 400이 아니라 `ValueError`로 바로 드러냅니다
`read_audio_info`를 먼저 호출	존재·형식 오류(404·415)를 180과 똑같이 맞추려고 재사용합니다. 헤더만 한 번 더 읽어 비용이 거의 없습니다

6. 시험 (`tests/test_sample_audio_service.py`에 15개 추가, 전체 899개 통과)

파일은 임시 폴더(`tmp_path`)에 만들었고, 샘플 값은 직접 넣어 확인합니다.

시험	확인한 것
길이와 정보	`peaks`가 `DEFAULT_BUCKETS`(1000)개이고, 값이 `(float, float)` 쌍이며, `sample_rate=44100`, `channels=1`

시험	확인한 것
무음	모든 칸 `(0.0, 0.0)`
일정한 0.5	10칸 모두 `(0.5, 0.5)`
사인	첫 칸이 음수·양수를 모두 가지고, 모든 값이 −1 ~ 1
스테레오 평균	`channels=2`, 모두 `(0.0, 0.0)`
칸 수	buckets=10 → 10칸, buckets=1 → 1칸
표본보다 큰 buckets	표본 10개 → 칸 10개로 줄임
buckets=1	파일 전체의 최솟값·최댓값 `(-0.5, 0.5)`
자름·반올림	2.0 → `(1.0, 1.0)`, 0.123456 → `(0.1235, 0.1235)`
다른 형식	FLAC도 같은 결과
buckets=0	`ValueError`
없는 파일 / 오디오 아님 / 빈 파일	404 / 415 / 400
값 묶음	`Waveform`이고 값이 바뀌지 않음 (frozen)